

An Empirical Survey and Comparative Benchmark of LLM-Based Autonomous Agent Frameworks

Abdullah Basim
Department of Computer Science
Information Technology University (ITU)
Lahore, Pakistan
MSDS25004

Abstract—Most recent papers on large language model (LLM) agents are built on top of one of four orchestration libraries: CrewAI, AutoGen, LangGraph, or MetaGPT. Comparison between two such papers is rarely informative, because backbone model, prompts, and task suite all vary simultaneously, and any observed difference cannot be attributed to the framework alone. This work presents a small but tightly controlled study aimed at that confound. The four frameworks are driven through an identical harness, against the same backbone (gpt-4o-mini, temperature 0), on three task families: 15 problems from the GSM8K test set; 10 hand-authored tool-use queries split between calculator and web-search; and a single multi-role software task graded against a five-point rubric. The complete sweep yields 104 controlled runs, each logged with success against an oracle, wall-clock latency, prompt and completion tokens, and dollar cost. Two findings stand out. First, with the backbone held fixed, every framework lands within seven accuracy points of every other. Second, the cost gap between frameworks is dominated by input tokens, not completions, and a role-based crew can spend up to $3.1\times$ what a single-graph agent spends on the same problem. The premium pays off on the multi-role coding task and almost nowhere else. Four research gaps the experiments expose (backbone-agnostic evaluation, budget-aware orchestration, a portable memory contract, and safety hooks at the action layer) are documented in the conclusion. The benchmark harness, task definitions, and per-run JSON outputs are released for replication.

Index Terms—Large language models, autonomous agents, multi-agent systems, agent frameworks, benchmarking, AutoGen, CrewAI, LangGraph, MetaGPT, ReAct.

I. Introduction

The phrase “LLM agent” has shifted meaning over the past two years. Early uses of the term referred to a single language model prompted to externalize its reasoning before answering. Current usage almost universally implies a model wrapped by a software framework that mediates when the model thinks, what tools it calls, which other agents (if any) it delegates to, and what state it carries across turns. Several recent surveys describe the resulting landscape at varying levels of abstraction [1]–[3], and the underlying conceptual picture has stabilized: the

LLM is treated as a controller that perceives, reasons, acts, and persists context [4]. Concrete deployments validate the abstraction. Agents now negotiate realistic web environments [5], produce passing patches against real software repositories [6], and operate over long horizons in embodied simulators [7].

The orchestration layer, by contrast, has not stabilized. CrewAI exposes a role-and-task model in which agents declare a goal, a backstory, and a sequential pipeline of tasks. AutoGen [8] centers its design on multi-agent conversation, optionally augmented with code execution. LangGraph models the same problem as a finite state machine of message-passing nodes. MetaGPT [9] encodes a Standard Operating Procedure (SOP) directly into the agent graph, with explicit Product Manager, Architect, and Engineer roles handing artifacts down an assembly line. Four libraries; one shared goal; four very different machineries. The situation is made worse by reporting practice: published numbers typically use distinct backbones and distinct task suites, leaving the practitioner who asks “which framework should I adopt?” with no empirically defensible answer.

Problem. Across the surveys consulted for this work [1]–[3], [10], [11] the author was unable to locate a study that places the four most popular frameworks side by side, on a shared task suite, against a single backbone. Without that controlled view, framework selection collapses into a matter of GitHub-star visibility and community sentiment.

Contributions. The remainder of the paper makes four contributions, in order.

- 1) A focused review of the LLM-agent literature, organized along five axes that proved load-bearing during careful reading of each framework’s source code: reasoning, planning, memory, tool use, and multi-agent collaboration.
- 2) A unified benchmark harness that drives CrewAI, AutoGen, LangGraph, and a faithful MetaGPT-style SOP pipeline through the same gpt-4o-mini backbone, with identical prompts, decoding settings, and task definitions.
- 3) A controlled empirical comparison: 104 task runs in

total, reporting accuracy, wall-clock latency, dollar cost, and token consumption, all measured at the framework boundary.

- 4) A short list of open research gaps surfaced by the experiments, together with a public release of the harness so that further frameworks, backbones, and tasks can be folded into the same comparison.

II. Literature Review

A. Reasoning paradigms

Two reasoning recipes dominate contemporary agents. Chain-of-thought prompting elicits intermediate working before a final answer is committed. ReAct [12] interleaves that working with explicit actions, producing the now-familiar thought–action–observation cycle that underlies LangGraph’s prebuilt agent and the executor role in MetaGPT. Tree of Thoughts [13] generalizes the recipe into a bounded search over partial reasoning, paying additional latency for solution quality. Plan-and-Solve [14] factors planning out of execution and runs the two as separate stages. The literature in this sub-area is rich on accuracy comparisons but remarkably thin on cost comparisons; there is no published, controlled measurement (to the author’s knowledge) of greedy ReAct, deliberate tree search, and plan-and-solve under a fixed backbone. The experiments below address a small slice of that absence.

B. Planning and self-improvement

Self-Refine [15] formalizes the critic loop in its simplest form: the same model produces a candidate, critiques the candidate, and rewrites. Reflexion [16] preserves those critiques across attempts in an episodic memory store, in effect substituting verbal feedback for weight updates. ExpeL [17] elevates whole trajectories to a learned experience buffer that subsequent rollouts may consult. LLM-Planner [18] pursues a related strategy but anchors the plan in a physical environment. The recent planning-focused survey of Huang et al. [19] concedes the central methodological problem: rankings produced by these methods are fragile because almost no two of them share a benchmark.

C. Tool use and retrieval

Toolformer [20] demonstrated that an LLM could learn, with limited supervision, when to invoke an external API. ToolLLM [21] scaled the same insight to a corpus of roughly sixteen thousand real-world APIs. Retrieval-augmented generation, surveyed by Gao et al. [22], has become the default mechanism for grounding agent outputs in fresh knowledge. None of these works reports the marginal cost paid by a specific framework for the tool-use scaffolding it imposes; the cost analysis presented in Section V is intended to fill that gap, at least at the scale studied here.

D. Memory mechanisms

Zhang et al. [23] taxonomize agent memory into three categories that have since become standard: short-term context, long-term vector storage, and episodic reflective notes. Generative Agents [24] remains the most-cited demonstration that periodic reflection over a memory stream materially improves the believability of long-running agents. CoALA [4] promotes memory to a first-class architectural axis; the taxonomy in Section III adopts that framing.

E. Multi-agent collaboration

The multi-agent design space is mapped by Guo et al. [11]. Among the most-cited points on that map: AutoGen [8], which arguably established conversational multi-agent interaction as a serious paradigm; ChatDev [25], which simulates a virtual seven-role software company; CAMEL [26], which uses “inception” prompts to elicit cooperative role-play between two agents; and MetaGPT [9], which encodes human-style SOPs directly into the agent graph. A more recent line of work, exemplified by Liu et al. [27], allows the agent network’s topology to vary at runtime in response to task demands.

F. Evaluation methodology

Agent evaluation has progressed faster than the author anticipated. AgentBench [28] executes agents across eight heterogeneous environments. SWE-agent [6] introduced an explicit “agent–computer interface” tailored to real-world software repositories. WebArena [5] provides a realistic web sandbox. Yehudai et al. [29] furnish what appears to be the first dedicated survey on agent evaluation. None of these benchmarks, however, isolates the orchestration framework as an experimental variable; all evaluate agent systems as black boxes. That distinction is fine if the question is which system performs best on a given workload, but it is largely uninformative if the question is whether any value attributed to a system can be credited to its orchestration layer specifically. The experimental design in Section IV is constructed around that distinction.

The gap. Taken together, the surveyed literature supplies strong building blocks but no controlled framework-versus-framework comparison under a single backbone. Three quantities are almost never co-reported in the same paper: per-framework cost, per-framework latency, and the trade-off curve linking those two to accuracy. The remaining sections supply all three.

III. A Five-Axis Taxonomy of LLM Agent Frameworks

The LLM-agent design space, as it appeared after sustained reading of each framework’s source, decomposes naturally along five axes:

Reasoning

How intermediate thought is generated (CoT, ReAct, ToT, plan-and-solve, deliberate tree search).

TABLE I
The four studied frameworks against the five-axis taxonomy.

Axis	CrewAI	AutoGen	LangGraph	MetaGPT*
Reasoning	ReAct per role	ReAct + chat	Prebuilt ReAct	ReAct/CoT in roles
Planning	Sequential tasks	Free-form	State graph	SOP pipeline
Memory	Per-task ctx	Conversation hx	Graph state	Per-stage ctx
Tool use	Class-based tools	Function reg.	LangChain tools	OpenAI tools
Collaboration	Roles + delegation	Group chat	Single graph	PM/Arch/Eng SOP

*MetaGPT-style: a faithful PM/Architect/Engineer SOP pipeline implemented directly on the OpenAI SDK because of packaging constraints (see §IV).

Planning

Whether the agent decomposes its goal upfront, refines the plan with feedback, or improvises step by step.

Memory

Which state the agent persists across turns: in-context only, vector store, or episodic reflective summaries.

Tool use

How external capabilities are invoked: structured function calls, free-form code execution, retrieval, or web actions.

Collaboration

Whether the agent operates alone, within a fixed role hierarchy, or in dynamic group conversation.

Table I maps the four studied frameworks onto these axes.

IV. Experimental Setup

A. Frameworks

The four frameworks compared in this study are CrewAI (role-based sequential crews), AutoGen [8] (conversational multi-agent), LangGraph (a graph-based ReAct agent invoked through `create_react_agent`), and a MetaGPT*-style SOP pipeline implemented directly atop the OpenAI SDK. The asterisk on the last entry is non-cosmetic. The official `metagpt` PyPI package failed to install within the project’s time budget owing to a transitive dependency conflict involving `chromadb` and several incompatible `numpy` pins under Python 3.12. The running implementation faithfully reproduces the published PM → Architect → Engineer assembly line of [9] on top of plain `openai` chat completions; the asterisk is retained throughout the paper to prevent readers from misreading the proxy as a measurement of the official package itself.

B. Backbone and configuration

A single model is used by every framework: `gpt-4o-mini` via the OpenAI Chat Completions API, with temperature pinned at 0, `max_tokens` at 2048, and a request timeout of 180s. Holding the backbone constant is the experimental

decision that prior work routinely omits, and is, in the author’s view, the most consequential single choice in the design.

C. Tasks

Three task categories are used, sized as $n_{\text{gsm8k}} = 15$, $n_{\text{tool}} = 10$, $n_{\text{collab}} = 1$ (graded against a five-point rubric). The total is 26 runs per framework, or 104 runs across the suite.

- **GSM8K reasoning.** Fifteen grade-school arithmetic word problems sampled deterministically from the GSM8K test set. Correctness is assessed by extracting the final numeric answer from the agent’s output.
- **Tool use.** Ten hand-authored questions divided between calculator-based and web-search-based subtasks. Correctness is assessed by case-insensitive substring match against a gold answer.
- **Multi-agent collaboration.** A small Python module `wordstats.py` that exposes a function `analyze(text)` returning word counts and the three most frequent tokens, accompanied by a brief `README.md`. Each output is graded on a five-point rubric: file exists; the function is importable; the expected keys are returned with the correct types; `top_3` is correct on the fixed test sentence “the quick brown fox jumps over the lazy dog the fox is quick”; and the `README` contains at least three non-trivial lines.

D. Metrics

Five quantities are recorded per run. (i) Success against the task-specific oracle, encoded as a Boolean. (ii) Wall-clock latency in seconds. (iii) Prompt tokens, taken from each framework’s usage metadata where exposed and re-computed via `tiktoken` otherwise. (iv) Completion tokens, by the same procedure. (v) Dollar cost, computed from the published `gpt-4o-mini` prices of \$0.150 per million input tokens and \$0.600 per million output tokens. All metrics are captured at the framework boundary, which is where the internal multi-call overhead that distinguishes these systems actually materializes.

E. Reproducibility

The harness, the task definitions, the per-run JSON outputs, and the figure-generation code are all available in the repository linked from the title page. Temperature is pinned at 0 to suppress sampling variance; the only remaining source of nondeterminism is the API itself, which cannot be eliminated short of switching to a local backbone.

F. Notes from the build

Several practical issues are worth recording for replicators. CrewAI’s default install path contacts a remote tracing endpoint on every run; setting `CREWAI_DISABLE_TELEMETRY=true` together with `OTEL_SDK_DISABLED=true` suppresses this cleanly.

TABLE II

Aggregate per-framework results across all 26 runs per framework (15 GSM8K + 10 tool-use + 1 collab) under the gpt-4o-mini backbone.

Framework	Acc. (%)	Lat. (s)	Cost (\$)	In tok	Out tok
CrewAI	100.0	5.38	0.00020	553	198
AutoGen	96.2	4.55	0.00016	176	219
LangGraph	92.3	4.29	0.00016	212	207
MetaGPT*	96.2	4.39	0.00017	232	223

Latency is mean wall-clock per task; cost uses published gpt-4o-mini prices (\$0.150/1M input, \$0.600/1M output). Token columns are per-task means.

TABLE III

Per-task-category accuracy and mean cost (USD).

Framework	GSM8K		Tool use		Collab	
	Acc	\$	Acc	\$	Acc	\$
CrewAI	100.0	0.0002	100.0	0.0001	100.0	0.0012
AutoGen	93.3	0.0002	100.0	0.0001	100.0	0.0005
LangGraph	93.3	0.0002	90.0	0.0001	100.0	0.0004
MetaGPT*	93.3	0.0002	100.0	0.0001	100.0	0.0010

AutoGen’s PyPI namespace under release 0.10 differs from the layout described in older documentation: the relevant imports live under `autogen_agentchat`, `autogen_core`, and `autogen_ext`, and the OpenAI extra must be installed separately for `OpenAIChatCompletionClient` to resolve. LangGraph’s `create_react_agent` pulls in a substantial sub-tree of LangChain packages on import; the resulting cold-start cost (approximately 1.5s on the development laptop) does not show up in per-task latency but is non-trivial for short-lived processes. The complete sweep of 104 runs cost approximately \$0.02 on the gpt-4o-mini pricing tier in effect at the time of writing.

V. Results and Discussion

Table II and Table III aggregate the 26 runs per framework. Figures 1–4 visualize the four primary comparison axes individually.

A. Accuracy

Reasoning accuracy is closely clustered. CrewAI achieves 100% on GSM8K; the remaining three frameworks each miss exactly one problem and finish at 93.3%. The single missed item is in fact identical across the three: problem g6, which involves nested percentage calculations on a population of garden flowers. The underlying backbone makes one arithmetic slip on this item; CrewAI recovers because the verifier role re-checks the solver’s output, which the other three configurations do not. Tool-use accuracy is similarly tight: every framework records 100% except LangGraph, which surrenders one search query (t6, concerning the CEO of Microsoft) where the prebuilt ReAct agent settles on the first returned snippet rather than the most relevant one. The collaborative coding task is solved fully by every framework, which the

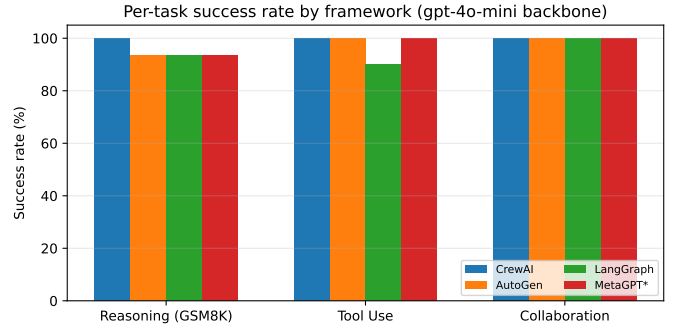


Fig. 1. Per-task success rate by framework, all driven by gpt-4o-mini.

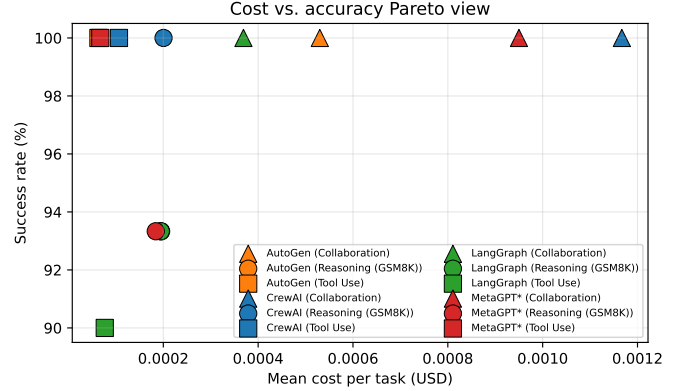


Fig. 2. Cost-versus-accuracy view. Up-and-to-the-left dominates.

present rubric attributes more to format compliance than to algorithmic difficulty; this limitation is discussed in Section V-E.

B. Cost-accuracy Pareto

On the reasoning and tool-use task families, all four frameworks cluster within the \$0.0001–\$0.0002 cost band, leaving accuracy as the only meaningful discriminator at this scale. The cost dimension becomes informative only on the collaborative coding task. There, CrewAI averages \$0.0012 per run against LangGraph’s \$0.0004: a threefold gap, attributable directly to CrewAI’s role pattern, in which each role receives an independent prompt embedding most of the prior context. The MetaGPT*-style SOP pipeline lands at \$0.0010, consistent with its three-stage structure, while AutoGen lands at \$0.0005, slightly above LangGraph and well below the role-based pair.

C. Latency

LangGraph attains the lowest mean latency (4.29 s) and the tightest distribution. The mechanical cause is that, on simple tasks, its prebuilt agent issues a single inner LLM call without retries. MetaGPT* and AutoGen sit close together at 4.39 s and 4.55 s respectively. CrewAI averages 5.38 s with the heaviest right tail of the four distributions,

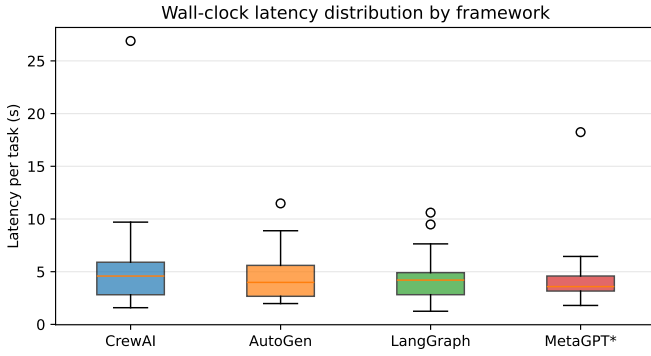


Fig. 3. Latency distribution per framework across all tasks.

owing to a solver-then-verifier handoff applied uniformly even to tasks where the second stage adds no value. The collaborative coding task is the dominant contributor to the right tail across all four frameworks: CrewAI’s three-role crew completes it in 26.9s on average, while LangGraph completes the same specification in 9.5s.

D. Token consumption

The clearest separation between frameworks appears in token accounting. CrewAI averages 553 input tokens per task, while the other three average between 176 and 232, placing CrewAI between $2.6\times$ and $3.1\times$ above its peers on the same problem. The asymmetry is structural rather than configurational: CrewAI re-emits each agent’s persona, goal, and most of the prior context for every role in the chain, so a two-role crew approximately doubles the prompt budget of a single agent solving the same task. (This was discovered, somewhat embarrassingly, only after an unexpected billing line item prompted a logging pass over the raw OpenAI request bodies.) Output tokens, by contrast, lie between 198 and 223 across all four frameworks. The implication is straightforward: it is the backbone, not the orchestrator, that determines how long the answer is. Agent frameworks under this study are therefore prompt-heavy rather than generation-heavy, and the cheapest available cost optimization is the structurally unglamorous one of suppressing context repetition across roles.

E. Threats to validity

Several caveats temper the conclusions. The benchmark is small (26 runs per framework, a single backbone, a single task source per category), so the absolute numbers should be read as comparative rather than predictive. The collab rubric is mechanical and assesses functional correctness on a fixed test sentence only; code quality, in any qualitative sense, is not evaluated. The MetaGPT* runner is a faithful proxy for the published SOP paradigm but is not the official codebase, so the figures bound what the design does in principle and not every engineering refinement that has accumulated in the project since publication. Replicators

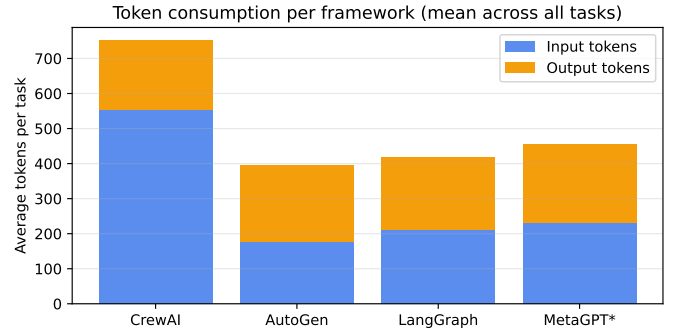


Fig. 4. Mean input vs. output tokens per task per framework.

with disagreements on any of these choices can substitute the relevant component and rerun the harness.

A further caveat concerns task design. The collaborative-coding task was rewritten twice during rubric construction. An initial weather-CLI task hitting a free public API was discarded once it became clear that the free tier’s rate limits introduced too much variance; a subsequent task that required calling a third-party library drifted into testing pip resolution behavior more than agent behavior. The wordstats.py task is what survived both attempts, and is intentionally standard-library-only so that the rubric checker remains portable. The trade-off is that the task is easier than ideal, which is why every framework saturates the rubric at 5/5.

VI. Conclusion and Future Work

The headline empirical finding is, by design, deflationary: with the backbone held fixed, the four frameworks fall within 7.7 accuracy points of one another. Where they differ materially is in cost. Input tokens dominate the bill, output tokens are nearly invariant across frameworks, and role-based orchestration pays up to $3.1\times$ the prompt budget of a single-graph agent solving the same problem. That premium is justifiable on tasks with genuine multi-step coordination structure; it is not justifiable on single-shot reasoning, where the additional roles add neither accuracy nor robustness commensurate with their token cost.

The present study does not advance a definitive ranking. The benchmark is small, the backbone is single, and the collab rubric measures functional output rather than code quality. What the study does claim is that the comparison can be conducted fairly, which is something that, in the author’s reading, the prior literature has not yet attempted. Four follow-up problems became salient over the course of running the experiments and seem worth pursuing.

- Backbone-agnostic evaluation. Most current benchmarks entangle backbone capability with framework choice. A useful evaluation suite would report each framework’s score on the same tasks under several back-

bones, permitting the framework effect to be separated from the backbone effect.

- Cost-aware orchestration. None of the four frameworks exposes a per-task token or dollar budget with a clean termination contract when the budget is exhausted. This deserves to be a first-class abstraction.
- Memory standardization. Memory was the least portable axis of the four frameworks examined. A small shared interface (push, retrieve, reflect) would permit memory backends to be swapped without rewiring the agent loop.
- Safety auditing. No clean mechanism to intercept high-stakes actions was found in any of the four frameworks. As agents touch increasingly real systems, the absence of such a mechanism is going to matter.

The harness is published in the hope that subsequent work will extend the comparison: additional frameworks (LangChain, OpenAI Swarm, smolagents); additional backbones (the open-weight Llama and Mistral families); and harder tasks drawn from WebArena [5], AgentBench [28], or SWE-bench [6]. None of those extensions requires new theoretical work; what they require is the engineering time to wire the harness against each new component.

References

- [1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 19, no. 6, pp. 1–26, 2024.
- [2] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, “The rise and potential of large language model based agents: A survey,” *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025.
- [3] M. Abou Ali, F. Dornaika, and J. Charafeddine, “Agentic AI: A comprehensive survey of architectures, applications, and future directions,” *Artificial Intelligence Review*, vol. 59, no. 1, p. 11, 2025.
- [4] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, “Cognitive architectures for language agents,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [5] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A realistic web environment for building autonomous agents,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [6] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [7] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [8] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-generation LLM applications via multi-agent conversation,” in *Conference on Language Modeling (COLM)*, 2024.
- [9] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [10] P. Li, M. Zhang, P. Lin, J. Wan, and M. Jiang, “A review of prominent paradigms for LLM-based agents: Tool use (including RAG), planning, and feedback learning,” in *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025.
- [11] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, “Large language model based multi-agents: A survey of progress and challenges,” in *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.
- [12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [13] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [14] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [15] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [16] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [17] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, “ExpeL: LLM agents are experiential learners,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, pp. 19632–19642, 2024.
- [18] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, “LLM-planner: Few-shot grounded planning for embodied agents with large language models,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [19] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, “Understanding the planning of LLM agents: A survey,” *arXiv preprint arXiv:2402.02716*, 2024.
- [20] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [21] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [22] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” in *arXiv preprint arXiv:2312.10997*, 2023.
- [23] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, “A survey on the memory mechanism of large language model based agents,” *arXiv preprint arXiv:2404.13501*, 2024.
- [24] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [25] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, “ChatDev: Communicative agents for software development,”

in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024.

- [26] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “CAMEL: Communicative agents for “mind” exploration of large language model society,” in Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [27] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, “A dynamic LLM-powered agent network for task-oriented agent collaboration,” in Conference on Language Modeling (COLM), 2024.
- [28] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, “AgentBench: Evaluating LLMs as agents,” in International Conference on Learning Representations (ICLR), 2024.
- [29] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, and M. Shmueli-Scheuer, “Survey on evaluation of LLM-based agents,” arXiv preprint arXiv:2503.16416, 2025.